Keywords: chain-of-thought • test-time compute • expansion utility • self-consistency

Expanding LLM Reasoning

Branching From the Last Step of an Existing Chain Beats Uniform Placement and Rivals Learned Routers While Using 0.77× the Tokens of Self-Consistency

By Rian Atri & Evan Luo

arXiv:2610.05584 • NeurIPS 2026 • October 2026

Instead of sampling multiple full chains from scratch, *Expanding LLM Reasoning* asks: given one completed chain-of-thought trace, at which step should an additional continuation branch? A simple "always branch from the last step" rule surpasses uniform placement across 41 model-and-benchmark cells and uses only 0.774× the generated tokens of four-sample self-consistency at matched accuracy on MATH-500.

AT A GLANCE

Problem: Self-consistency generates full chains from scratch; it ignores information already encoded in a completed trace.

Why it matters: Inference compute is the bottleneck for large reasoning models; reusing partial traces can reduce cost.

Method: Define expansion utility at each step; compare uniform, always-last, and learned-router branching policies across 41 cells.

Result: Always-last matches self-consistency accuracy using 0.774× the tokens on MATH-500; +4.25-point oracle headroom motivates future learned selectors.

Evidence: Nine models, six benchmarks, cross-fitted held-out evaluation.

The Big Picture

Chain-of-thought (CoT) reasoning substantially improves LLM accuracy on hard tasks, but generating complete independent chains is expensive. Self-consistency and best-of-n approaches sample n full chains, consuming $n\times$ the output tokens.

This paper reframes the problem: if one chain τ is already in hand, at which step t^* should a continuation branch to maximally improve the probability of reaching the correct answer? The key insight is that later steps encode more context and have lower continuation variance—making the last step a reliable default branching point regardless of the task or model.

Why Existing Approaches Fall Short

Self-consistency samples complete chains from scratch and majority-votes. It discards the partial trace information already computed.

Best-of-n with process reward models requires training a step-level critic, which is expensive and brittle across domains.

Learned branching routers require training data and do not consistently beat a simple positional heuristic, as this paper shows.

Core Mechanism: Expansion Utility

The authors define **expansion utility** at step t as the expected correctness improvement from branching at t:

$$U(t) = \mathbb{E}_{c\sim p(\cdot|\tau_{:t})}[\mathbf{1}[a(c) = a^*]] - \mathbf{1}[a(\tau) = a^*]$$

Positive $U(t)$ means branching at t is expected to recover a failed chain.

Three branching policies are compared:

- **Uniform:** branch at a randomly selected step.
- **Always-last:** always branch at the final step $|\tau|$.
- **Learned router:** a 2-layer MLP predicting $U(t) > 0$ from prefix hidden states.

Technical Formulation

Let $\tau = (s_1, \ldots, s_T)$ be a chain and $\tau_{:t}$ its prefix. With k continuations sampled from step t, the expected accuracy is:

$$\mathrm{Acc}(t, k) = \mathbb{E}_{c_1,\ldots,c_k\sim p(\cdot|\tau_{:t})}[\mathrm{MajVote}(a(c_1), \ldots, a(c_k)) = a^*]$$

The optimal policy selects $t^* = \arg\max_t \mathrm{Acc}(t, k)$.

For the learned router, let $\phi(\tau_{:t}) \in \mathbb{R}^d$ be the model's last-layer hidden state at step t. The router solves:

$$\hat{y}(t) = \mathbf{1}[\mathbf{w}^\top \phi(\tau_{:t}) + b > 0]$$

trained with binary cross-entropy on labels $\mathbf{1}[U(t) > 0]$.

Held-out expansion utility at selected step t^*:

$$\Delta\mathrm{Acc}(\pi) = \mathrm{Acc}(\pi(t), k) - \mathrm{Acc}(\mathrm{Uniform}, k)$$

measures the gain of policy π over the uniform baseline.

Learning or Inference Procedure

1. Generate one chain τ per problem instance.
2. Apply the branching policy to select step t^*.
3. Sample k=4 continuations from $\tau_{:t^*}$ in parallel.
4. Majority-vote over k answers.

5. Compare total tokens against self-consistency at matched accuracy.

The router is trained once per model on 1,000 labeled chains; evaluation uses cross-fitting (router trained on split A, evaluated on split B).

What the Guarantee Says

The paper establishes empirical monotonicity of variance across steps: later steps show lower continuation-correctness variance than earlier steps across 38/41 model-benchmark cells. This explains why always-last is a robust heuristic even without a learned predictor.

Oracle selection outperforms always-last by +4.25 points on average, showing residual headroom that better future selectors could capture.

Experimental Findings

- **Models:** DeepSeek-R1-Distill-Qwen-14B, Qwen3-14B, Gemma-2-9B, QwQ-32B, five smaller models.
- **Benchmarks:** MATH-500, AIME 2024/25, AMC 2023, GPQA-Diamond, LiveCodeBench.
- Always-last beats self-consistency at **0.774× tokens** on MATH-500 and 0.81× on AIME 2024.
- Learned router shows +4.25 gain on held-out cross-fitted sets vs. uniform, but not over always-last.
- Always-last beats uniform in 38/41 cells; ties in 3.

Ablations and Interpretation

Branch depth: Last vs. second-to-last step gives similar accuracy; the last step carries the most cumulative context.

Continuation length: Full-chain continuations outperform few-step endings slightly, suggesting the model benefits from regenerating the answer tokens.

Number of branches k: Accuracy improves log-linearly up to k=8; diminishing returns beyond.

Router architecture: 2-layer MLP outperforms linear probes; 4+ layers overfit the training split.

Why always-last works: Empirical $U(t)$ variance decreases monotonically with step depth; the last step minimises continuation variance, making it the most stable branching point.

Reference

Rian Atri, Evan Luo. **Expanding LLM Reasoning**. NeurIPS 2026. arXiv:2610.05584, October 2026.
https://arxiv.org/abs/2610.05584